Comparing Deep Learning Models for Burial Mound Segmentation Using Lidar Data

Friso Harff

7526946

Prof. dr. ir. Jan Broersen

Prof. dr. Rosalie Iemhoff

June 26, 2026

bachelor Kunstmatige Intelligentie, UU

7.5 EC

Abstract

This study applies and systematically compares four deep learning models for segmentation based on convolutional neural networks and lidar-derived data to segment burial mounds in the Netherlands. The models are trained on fifteen combinations of four different input layers. Data augmentation is used to improve the performance of the models.

The lidar data used were Digital Terrain Models from AHN4. From the DTMs, three derivations were made. The ground truth was drawn by hand for supervised learning. Images were split into smaller tiles for better training. Only tiles with 200+ positive pixels were used for training and testing. Four different model architectures were trained and tested using all combinations of the four lidar-derived data as input. Performance was measured in Intersection over Union, Dice Similarity Score, precision and recall. The models are custom implementations of Unet, TransUNet, DeepLabV3+, and DenseTrans. They are made lighter to be able to train on a small GPU.

This study shows that DeepLab has the highest precision. For recall, Unet scores highest. DeepLab scores slightly higher than Unet on IoU and DSC. TransUNet and DenseTrans both had lower performances. The best models all contained the Simple Local Relief Model as input channel. The worst models existed mostly of DTM used alone. The difference of computational costs between the models is not too large to consider choosing a cheaper model. I conclude that precision and IoU are the important performance measures for this application. Therefore, I conclude that DeepLab is the most suited. The most valuable input channel is SLRM.

Transformer-based models performed worse than Convolutional Neural Networks. This could be due to having little training data, not enough transformers, or transformers not being fit for the task.

All code can be found on https://github.com/YenotyEtoKruto/KIThesis.

Table of Contents

Introduction 4
Methodology 7
Data Collection 7
Dataset Construction 7
Training 8
Analysis 9
Models and Implementation 11
Unet 11
TransUNet 12
DeepLabV3+ 15
DenseTrans 17
Results 19
Performance 19
Computational Costs 21
Input Channels 22
Discussion 25
Architectural Differences and Model Performance 25
Computational Costs 26
Comparing Performance Metrics 27
Effect of Input Channels 27
Implications 28
Limitations 29
Recommendations 30
Conclusion 31
Code availability 33
References 34
Appendix A 40
Appendix B 44
Appendix C 46

Introduction

Cemeteries for burial purposes have been used by societies throughout most of history. One type of cemetery is the burial mound, which has been used across the globe, including the Netherlands (Karpińska, 2026; Maghsoud et al., 2026; Mäkinen et al., 2026; Miller & Rivera, 2006; Wicke, 1965; Wu et al., 2026). These mounds contain interesting and important information for studying old societies. They provide insights into ancient demography, the structure and organization of these societies, as well as their perspectives on death and burial practices (Bailey, 2002; Wu et al., 2026). Burial mounds in the Netherlands often come in clusters, particularly in ice-pushed ridges, like near the southern part of the Veluwe. Burial sites were often in the immediate proximity of settlements, or along main travel routes (Fontijn et al., 2010; Hupperetz et al., 2009).

Due to their widespread distribution, archaeologists have started using remote sensing techniques for locating burial mounds, as remote sensing allows for non-invasive detection (Mäkinen et al., 2026; Oltean, 2013; Sobotkova et al., 2024; Yang, 2025). Remote sensing is the collection of information about the Earth using aircraft and satellites (Cambridge University Press and Assessment, 2026). It is already being used in other fields as well, such as agriculture, where it is used for crop monitoring, soil assessment and pest detection (Victor et al., 2024; Yang, 2025).

One of these remote sensing techniques is light detection and ranging (lidar). Lidar is similar to radar, except it works with lasers instead of radio waves (Collis, 1969). It has existed for over half a century and has contributed to many archaeological discoveries (Vinci et al., 2024; Yang, 2025). One of the advantages of using lidar is that it can penetrate vegetation. This way forested areas can be searched better (Inomata, 2024).

Previous studies have already concluded that lidar can be used for archaeology in combination with deep learning models to segment archaeological sites (Guyot et al., 2021;

Vinci et al., 2024; Yang, 2025). Some of these models are Unet and TransUNet. Unet uses an autoencoder structure that first encodes the image to learn important features and later reconstructs the image in a decoder. This makes it good for finding local structures, but not as good for long-range dependencies. TransUNet adds transformers to model relationships across a more global plane next to the local structures (Chen et al., 2024; Ronneberger, 2015; Yang, 2025). However, systematic comparisons of deep learning models applied to lidar-derived data remain limited (Argyrou & Agapiou, 2022; Yang, 2025).

This study will focus on filling the gap of knowledge about what deep learning models are well suited for burial mound segmentation and why by answering the question: which model would be best suited for the application of segmentation of burial mounds? This question will be answered by first answering the following questions: what are the differences in the models, how do the differences affect the performances, what combination of input layers increases performance most, and what trade-offs of different performance metrics makes the best performing model?

These questions will be answered by extending on the work of Yang (2025) by systematically comparing the accuracy of DeepLabV3+ and DenseTrans, next to Unet and TransUNet.

DeepLabV3+ uses Atrous Spatial Pyramid Pooling, which helps account for different object scales. This can help with segmenting burial mounds of varying sizes. Because of this, I expect that DeepLabV3+ will identify more sizes of burial mounds, and therefore will classify less pixels falsely as background (Chen et al., 2016; Chen et al., 2017; Chen et al., 2018).

DenseTrans uses, next to an encoder-decoder structure, features across multiple layers, decreasing information loss, and uses transformers for more global features. This way DenseTrans combines local, global and multi-level features. I expect this model to

outperform Unet and TransUNet in performance, as similar results in different applications have already been produced (Kumar, 2025; ZongRen et al., 2023).

The models will be evaluated using one to four layers of input data to identify key features for selecting an appropriate model for segmentation of lidar-derived data.

Methodology

Data Collection

For the construction of a training dataset, lidar data was acquired from the Actueel Hoogtebestand Nederland (2025). The data used was from the set AHN4. This data consists of a Digital Terrain Model (DTM). A DTM consists of scans of the ground from high objects, like planes or satellites, that are filtered to remove local higher structures like trees, to acquire more insight about the elevation of the ground. The locations of identified burial mounds came from the Rijksdienst voor het Cultureel Erfgoed (2018).

Dataset Construction

The DTM data were preprocessed in Python 3.8. Three more different models for data were created. The first is hillshade, which uses elevation to recreate the effect of the sun creating shades, placing the “sun” at 315 degrees azimuth and 45 degrees altitude. Second is slope, where the slope in degrees is calculated from local differences in elevation using the Horn algorithm for slopes (Horn, 1981). Both hillshade and slope can highlight local differences in terrain, revealing local structures. Both are applied on the DTM using the osgeo-gdal package. Then a Simple Local Relief Model (SLRM) was made from the DTM using the rvt-py package. The SLRM highlights low relief structures like burial mounds in areas of flat to moderate topography (Kokalj, 2025; Zakšek et al., 2011). The mask layer was then made by drawing polygons over the burial mounds in QGIS 3.44 Solothurn.

In python, using the rasterio package for TIFF files, the large layers will be tiled into 256 x 256 pixel tiles. Only tiles with 200 or more positive pixels are used. This increases

exposure to the target, improving the models' performance of recognizing the burial mounds. If they were exposed predominantly to data without burial mounds it would cause a class imbalance, which would make the models perform worse (Kochkarev et al., 2020; Ramyachitra & Manikandan, 2014).

The tiles are in alphabetical order greedily divided into 3 subsets with a ratio of 7:2:1 for training, validation, and testing respectively. All input layers are normalized before training to a common scale.

Training

The models Unet, TransUNet, DeepLabV3+ and DenseTrans are trained using the exact same training and validation set of the data and the performance is tested on the exact same test set. This is done for all 15 combinations of 1, 2, 3 and 4 stacked layers. Data augmentation is applied during training, which consists of rotations and horizontal and vertical flips.

A weighted binary cross-entropy loss function, L_{WBCE}, together with a Dice loss function, L_{Dice}, are used to optimize the segmentation performance. Dice loss is region-based, while cross-entropy is distribution based (Kato & Hotta, 2023). The weighted binary cross-entropy loss function penalizes pixel-wise wrongly classifications, while the Dice loss function optimizes region-overlap and is less vulnerable to class imbalance. Combining these losses creates a trade-off, L_{combo}, that in other works performs better than either loss used individually (Taghanaki et al., 2019). The formulas for the separate losses and the combination loss are as follows:

$$L_{Dice} = 1 - \frac{2\Sigma_i y_i \hat{y}_i + \epsilon}{\Sigma_i y_i + \Sigma_i \hat{y}_i + \epsilon}$$

$$L_{WBCE} = -\frac{1}{N}\Sigma_i[w_p y_i log(\hat{y}_i) + (1 - y_i) log(1 - \hat{y}_i)]$$

$$L_{combo} = 0.5L_{Dice} + 0.5L_{WBCE}$$

With y_i denoting the ground truth label, $\hat{y}_i$ the predicted probability, and smoothing term $\epsilon = 10^{-6}$. The positive class weight was set to $w_p = 10$, addressing the strong class imbalance, as 96.53% of the pixels in the training dataset are still negative.

Analysis

The performance can then be measured in intersection over union (IoU), precision, recall and dice similarity coefficient (DSC) following the paper by Yang (2025). DSC describes the ability of the model to correctly identify if a burial mound is in the image, while IoU describes how well the model finds the area of interest in the image (Taciuc et al., 2025). Precision is the ratio of correct positive predictions out of all positive predictions. Recall is the ratio of correct positive predictions out of all true positive cases. These can be described using the four parts of a confusion matrix. A confusion matrix is a 2x2 matrix, where the ground truth in columns is compared with predictions in rows. This matrix contains four fields. First are true positives (TP), where a predicted positive case overlaps with the ground truth, and true negatives (TN), with the same for negative cases. Then there are cases where a negative prediction is made while the ground truth is positive, this is a false negative (FN), and the same for a positive prediction in a negative ground truth is a false positive (FP). The metrics can be expressed as follows in these terms:

$$IoU = \frac{TP}{TP + FP + FN}.$$

$$DSC = \frac{2TP}{2TP + FP + FN}.$$

$Precision = \frac{TP}{TP + FP}$.

$Recall = \frac{TP}{TP + FN}$.

Additionally, computational costs are measured using the model size in MB, training time in seconds, and floating point operations (FLOPs). Floating point operations are operations where addition, subtraction, multiplication, or division on one or more floating point numbers. Computational cost is necessary for fair comparison of the models, because a much more expensive model might slightly outperform a significantly cheaper model. This could make the cheaper model more interesting to expand on for application than the expensive model.

The effect of input channels is analysed by comparing the frequency of the channels in the best and worst models per architecture per performance metric.

For training, 100 epochs with a learning rate of 0.001 and batch size 8 will be used. The computations will be done using CUDA 11.2 and CuDNN 8.1.1 on a GeForce RTX 4060 6GB.

Models and Implementation

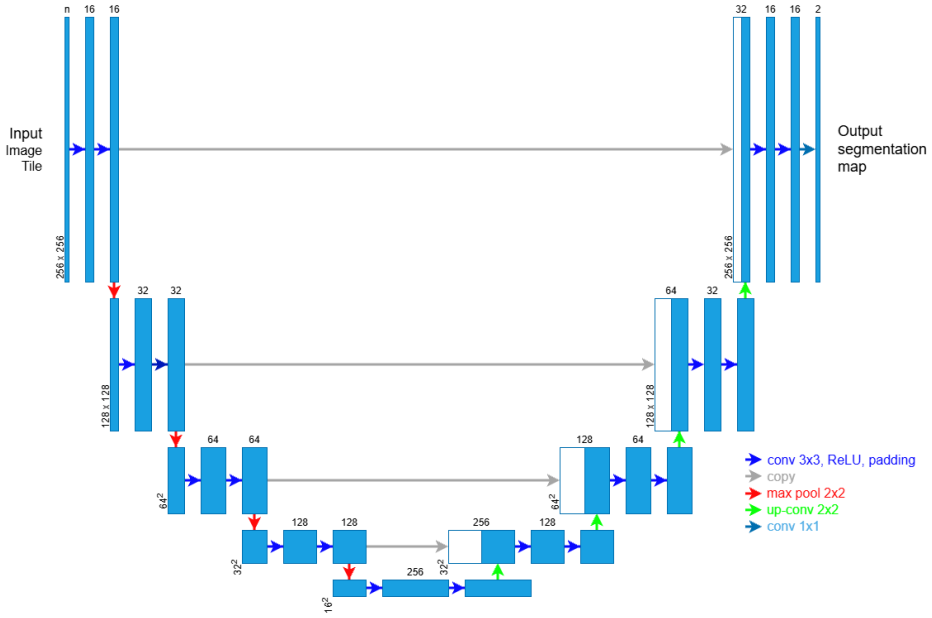
The models used in this research are slightly altered from the original models for application using limited GPU resources, but the architectures are kept faithful to the innovative concepts to allow for comparison.

Unet

The first model to discuss is Unet. Unet consists of three main parts: the encoder, the bottleneck, and the decoder, which when depicted looks somewhat similar to a U. In the encoder, feature maps are extracted from the input image using convolutions, ReLU and max-pooling. The bottleneck forms the main link between the encoder and decoder with only convolutions. Then in the decoder the image is reverse-engineered into a segmentation map from the feature map using up-sampling and convolutions. The reconstruction in the decoder is assisted by skip connections from the encoder, which help the decoder with spatial information that might have been lost in shrinking down the image. Because of the contracting and expanding structure of Unet, it is well-suited for learning local structures. Something done differently from the original proposal of Unet (Ronneberger, 2015) is the use of a smaller input, 256^2 instead of 572^2, and learning less features, with only 256 feature channels in the bottleneck opposed to the original 1024. The topology of Unet with the described changes is visualized in the diagram in Figure 1.

Figure 1

Topology of the custom Unet



Note. The topology of the custom Unet as used for comparison. From left to right, it consists of an encoder, a bottleneck, and a decoder. The encoder shrinks the image while growing the feature map to the bottom layer. The bottleneck only performs some convolutions. The decoder reconstructs the image using the feature map from the bottleneck and skip connections from the encoder.

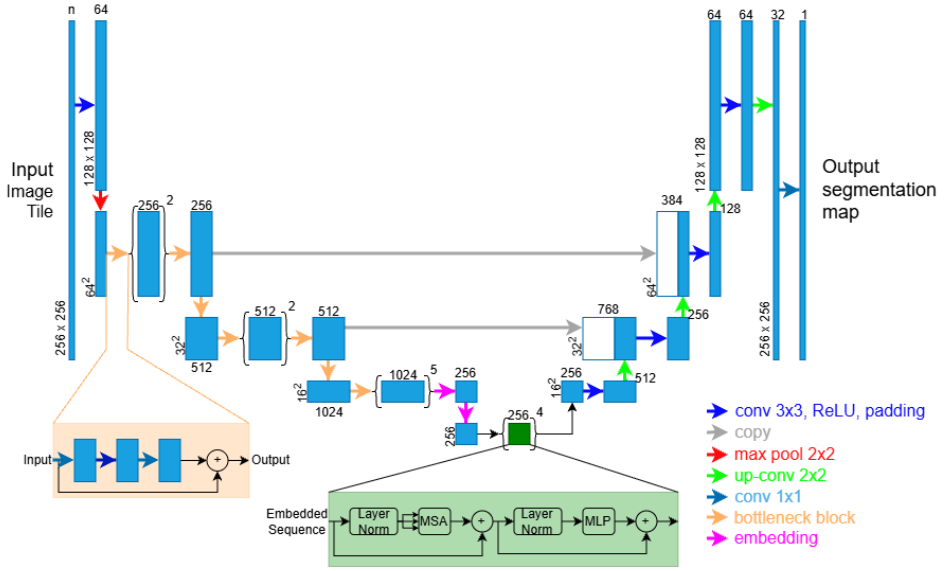
TransUNet

Unet is a model suited for local structures, but it lacks the ability to model relationships across a larger scale. This is useful, as most burial mounds come in clusters. For these clusters, the model could provide from the more global relations to recognize more mounds. TransUNet builds on the encoder-decoder structure of Unet by using transformers

instead of a bottleneck. In the proposal of TransUNet, the network consists of a hybrid CNN-transformer encoder and a decoder. For the CNN part of the encoder, a residual network called ResNet50 is used to create a feature map from the input image. This feature map is then tokenised into patches of 16, which are used as inputs for the Vision Transformer (ViT-B/16). The vision transformer then compares all the patches with each other, and the similarity between patches is measured by the self-attention. This way all parts of the picture are compared, and relationships are learned. This allows vision transformers to learn global relationships. The decoder reconstructs the image from the features extracted by the transformer using upsampling and convolutions, where skip connections allow for better reconstruction and stability. The addition of vision transformers and their self-attention mechanism should improve modelling of global relations, while the rest of the structure still allows for local feature learning (Chen et al., 2024; Dosovitskiy et al., 2020). The use of a residual network allows for easier optimizations and can improve accuracy from increased depth (He et al., 2016). The implementation of TransUNet used in this research is slightly altered from the original to accommodate the limited computing capacity. There are only 4 transformer blocks used, instead of 12. The hidden dimensions were reduced from 768 to 256 and for the Multihead Self-Attention only 4 attention heads were used instead of 12. For the MultiLayer Perceptron 768 dimensions are used, where the original used 3072. Because the input size is different, the amount of tokens that are put into the transformer are 16^2 instead of 14^2. The topology of TransUNet with the described changes is visualized in the diagram in Figure 2.

Figure 2

Topology of the custom TransUNet



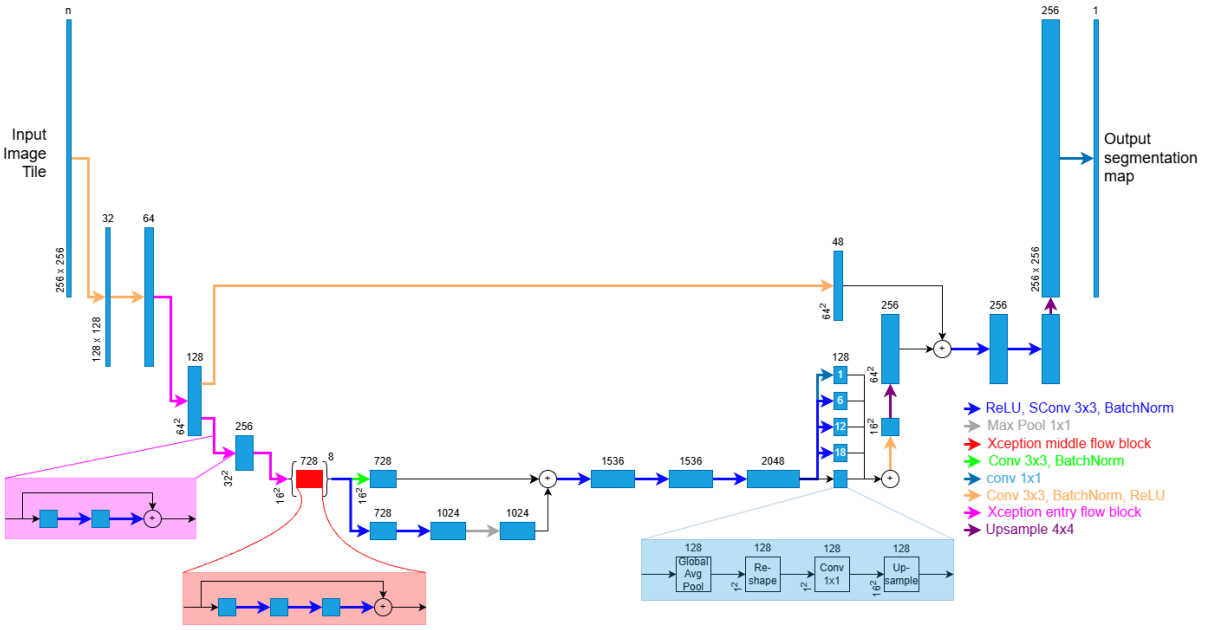
Note. The topology of the custom TransUNet implemented for comparison. It is composed of an encoder, transformers, and a decoder. The encoder applies a residual network backbone, highlighted in orange, to shrink down the image while growing the feature map. The transformers, highlighted in green, learn patterns of this feature map using self-attention. The decoder then uses this to reconstruct the image, aided by skip connections from the encoder.

DeepLabV3+

DeepLabV3+ is an extension to earlier models of the DeepLab-family. DeepLabV3+ uses a slightly modified Xception-65 as encoder, Atrous Spatial Pyramid Pooling takes the high-level features from the backbone and the result is put in a simple decoder which returns the image size to the input size. Xception-65 is a residual convolutional neural network that uses separable convolutions to reduce computational costs. It consists of an entry flow, a middle flow that can be repeated, and an exit flow (Chollet, 2016). Atrous Spatial Pyramid Pooling (ASPP) uses multiple parallel atrous convolutions with different atrous rates that are fused together, together with a global average pooling. Atrous convolutions can change the receptive field for filters based on the atrous or dilation rate. By combining multiple atrous convolutions with different dilation rates, ASPP combines features of different scales in a way a normal convolution could not achieve. There exists an issue where a high dilation rate causes an atrous 3x3 convolution to effectively become a 1x1 convolution, as the non-center points are spaced too far away to cover any important features. The global average pooling improves ASPP by compensating for this issue by providing global information. This helps account for different object scales. The decoder is added to refine the segmentation results along the boundaries of objects (Chen et al, 2016; Chen et al, 2017; Chen et al, 2018). The implementation of DeepLabV3+ used for this research consists of a smaller Xception backbone, with only 8 repetitions of the middle flow instead of 16. Further references to this implementation will be done by the name of DeepLab. The topology of DeepLab with the described changes is visualized in the diagram in Figure 3.

Figure 3

Topology of the custom DeepLab



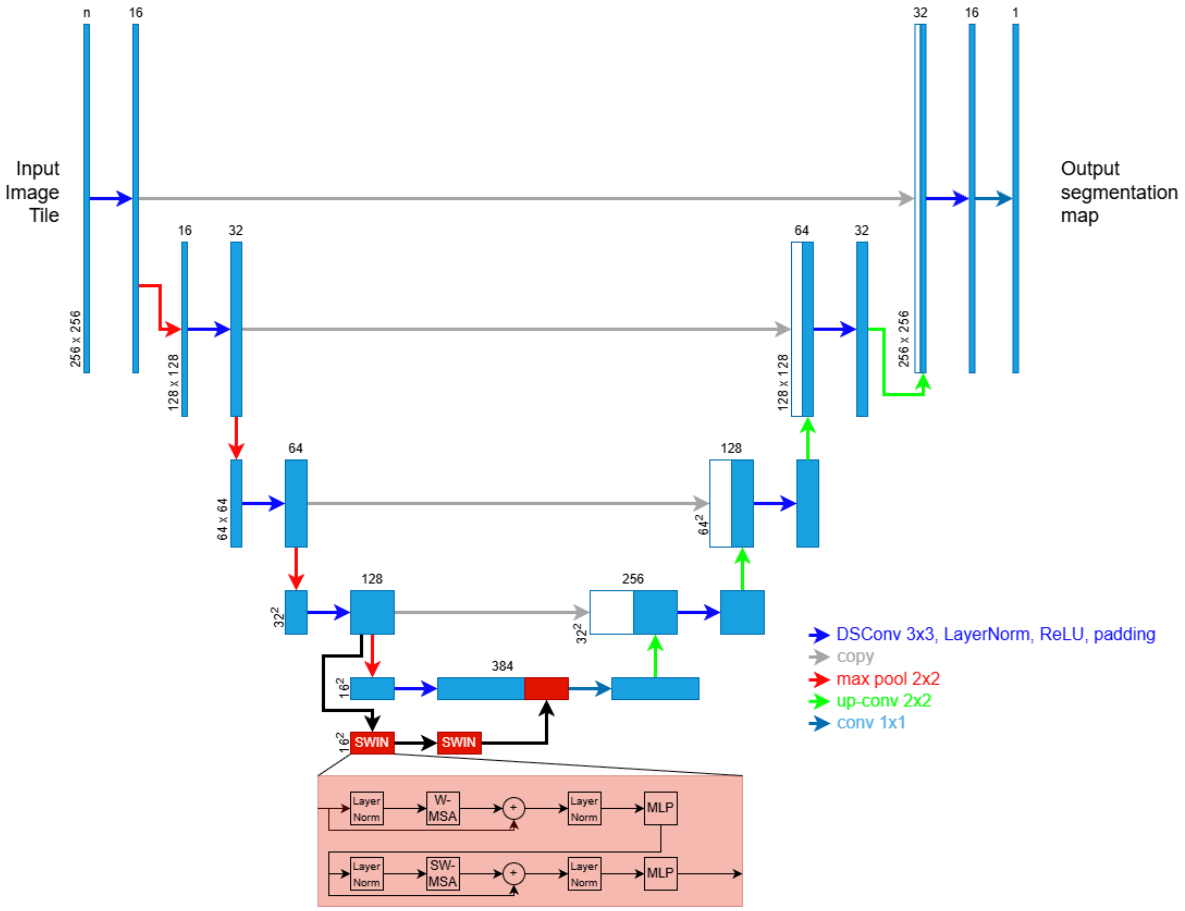
Note. The topology of the custom DeepLab implemented for comparison. This model consists of an encoder, Atrous Spatial Pyramid Pooling, and a decoder. The encoder is a smaller version of the residual Xception-65 model, with fewer middle flow blocks. The repeated part of the entry flow is highlighted in pink. The middle flow blocks are highlighted in red. The ASPP uses multiple parallel atrous convolutions next to a global average pooling, highlighted in light blue. The ASPP combines these and the decoder then reconstructs the image, using the feature maps from ASPP and a skip connection from the encoder, by upsampling the shrunken image.

DenseTrans

DenseTrans is another transformer model, just like TransUNet, but it uses a different type of transformer. Instead of the ViT it uses Swin transformer blocks. Swin transformer blocks do not function as global transformers, but instead first splits the input into different smaller windows before further splitting the windows in patches, and computes the Window Multihead Self-Attention per window (W-MSA). These windows contain patches of features that the CNN encoder of DenseTrans provide. Then in the second half of a Swin transformer block a Shifted Window Multihead Self-Attention (SW-MSA) is used, where the windows are shifted such that neighbouring patches that were in different windows in W-MSA are now mixed in the windows in SW-MSA to learn their relations as well (ZongRen et al., 2023). For the implementation two Swin transformer blocks were used, and the output is reshaped and concatenated with the output of the bottleneck before being passed on to the decoder. Adding more Swin transformer blocks created out of memory errors during training, so only two were used. The topology of DenseTrans with the described changes is visualized in the diagram in Figure 4.

Figure 4

Topology of the custom DenseTrans



Note. The topology of the custom DenseTrans implemented for comparison. It consists of an encoder, a bottleneck, Swin transformers, and a decoder. The encoder and decoder function similarly to those of Unet. The bottleneck output gets combined with the output of the Swin transformers, one such Swin transformer block is highlighted in red, which are parallel to each other.

Results

The results from the experiments are presented in three different sections. The first section is about how the models performed in terms of intersection of union, precision, recall and dice-coefficient. The second section summarizes the computational costs per model. The third section summarizes performance results regarding input channels. The complete performance results of all 60 unique models are reported in Appendix A: Performance Results per Model. The computational costs of all 60 unique models are reported in Appendix B: Computational Costs per Model.

Performance

The performance metrics are displayed in Figure 5. The highest and lowest maxima, minima and medians per statistic are as follows:

For intersection over union, TransUNet has the highest maximum, while DenseTrans has the lowest (0.591 vs. 0.530). DeepLab has the highest median, while DenseTrans has the lowest (0.543 vs. 0.491). DeepLab also has the highest minimum, and TransUNet has the lowest (0.425 vs. 0.049).

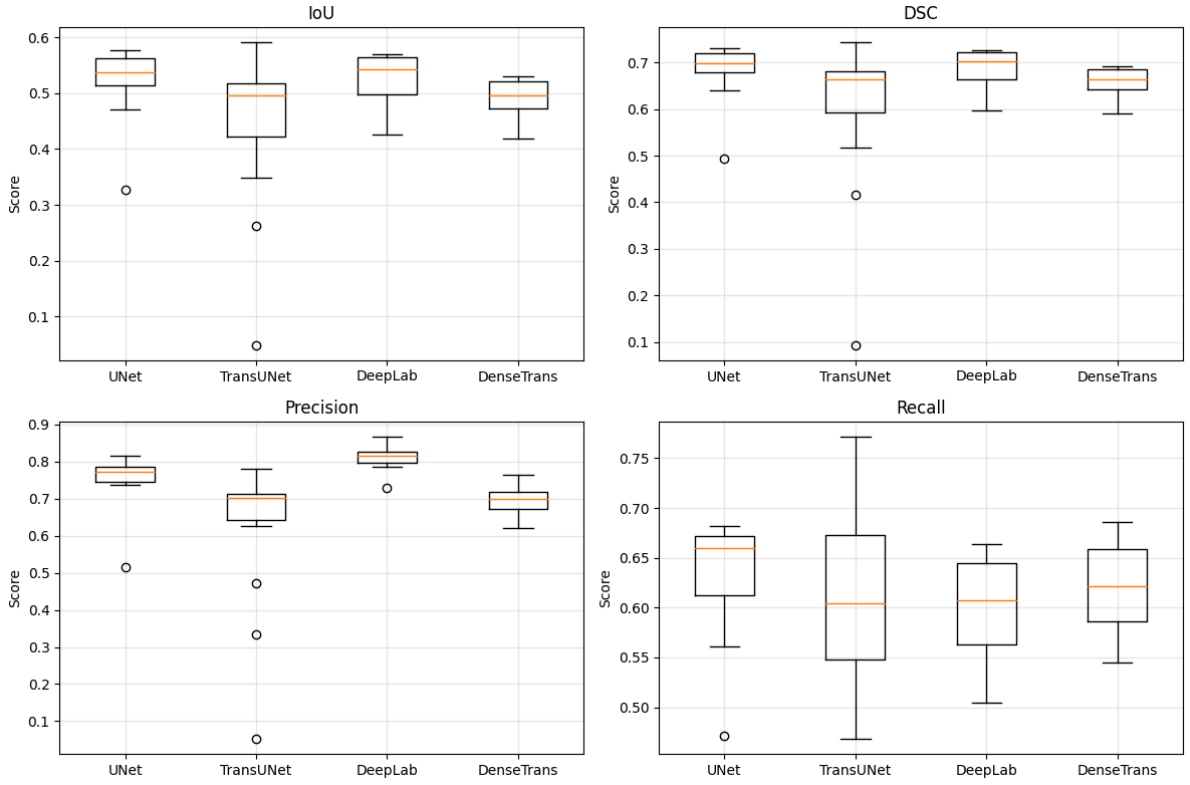
For DSC, TransUNet has the highest maximum, and DenseTrans has the lowest (0.743 vs. 0.693). DeepLab has the highest median, and TransUNet has the lowest (0.703 vs. 0.664). DeepLab has the highest minimum, and TransUNet has the lowest minimum (0.597 vs. 0.093).

For Precision, DeepLab has the highest maximum, and DenseTrans has the lowest (0.867 vs. 0.765). DeepLab also has the highest median, and DenseTrans also has the lowest (0.815 vs. 0.699). DeepLab also has the highest minimum, and TransUNet has the lowest (0.729 vs. 0.052).

For Recall, TransUNet has the highest maximum, and DeepLab has the lowest (0.772 vs. 0.664). Unet has the highest median, and TransUNet has the lowest (0.660 vs. 0.604). DenseTrans has the highest minimum, and TransUNet has the lowest (0.545 vs. 0.468).

Figure 5

Performance Metrics Across All Models



Computational Costs

The computational costs of the models are reported in Table 1. Unet is the fastest training model, while DenseTrans takes the longest (1037.80 ± 15.43 vs. 4166.20 ± 19.66 s). DenseTrans is the smallest in terms of model size, where DeepLab is the largest (17.81 ± 0.0 vs. 285.82 ± 0.0 MB). Model size is determined by the number of parameters and the number of bytes per parameter. DenseTrans has the least parameters, and DeepLab the most (1.51 vs. 24.93 million parameters). Unet has the smallest amount of floating point operations (FLOPs), while TransUNet has the largest amount (6.06 ± 0.02 vs. 20.52 ± 0.09).

Table 1

Summary of Computational Costs per Model

Model	Training Time (s)	Model Size (MB)	GFLOPs
Unet	1037.80 ± 15.43	22.42 ± 0.0	6.06 ± 0.02
TransUNet	3020.45 ± 17.57	190.28 ± 0.03	20.52 ± 0.09
DeepLab	2106.43 ± 14.19	285.82 ± 0.0	14.89 ± 0.01
DenseTrans	4166.20 ± 19.66	17.81 ± 0.0	—

Note. Means and standard deviation are reported.

— = FLOPs were determined using the TensorFlow profiler for Unet, TransUNet, and DeepLab. Due to the implementation of the Swin transformers in DenseTrans standard functions did not work on this model.

Input Channels

The best input channels per model per performance measure are summarized in Table 2. The worst input channels per model per performance measure are summarized in Table 3.

SLRM is included in all best performing models, most often accompanied by slope. Less than half of the best models also include DTM as an input channel, and only two contain hillshade.

DTM is in all worst performing models, most frequent when used alone. Second most frequent is the combination of DTM and hillshade. Slope is only included in one worst performing combination and SLRM in none.

Table 2

Best Input Channel Combinations per Performance Measure per Model

Model	Metric	DTM	Hillshade	Slope	SLRM
Unet	IoU	x		x	x
	DSC	x		x	x
	Precision	x		x	x
	Recall			x	x
TransUNet	IoU			x	x
	DSC			x	x
	Precision		x		x
	Recall			x	x
DeepLab	IoU	x			x
	DSC	x			x
	Precision				x
	Recall		x		x
DenseTrans	IoU			x	x
	DSC			x	x
	Precision	x		x	x
	Recall			x	x
Total number		6	2	11	16

Table 3

Worst Input Channel Combinations per Performance Measure per Model

Model	Metric	DTM	Hillshade	Slope	SLRM
Unet	IoU	x			
	DSC	x			
	Precision	x			
	Recall	x			
TransUNet	IoU	x			
	DSC	x			
	Precision	x			
	Recall	x			
DeepLab	IoU	x			
	DSC	x			
	Precision	x			
	Recall	x			
DenseTrans	IoU	x	x		
	DSC	x	x		
	Precision	x	x		
	Recall	x	x	x	
Total number		16	4	1	0

Discussion

The question this study tried to answer is: which deep learning model is best suited for burial mound segmentation using lidar data? Overall, the results indicate that DeepLab is the best suited for this application, as it has the highest median performance in IoU and precision, while being stable across all different performance metrics and input channel combinations. Some TransUNet models had higher scores for IoU and precision, but overall TransUNet performed worse than all other models. The costs of DeepLab are not enough to consider cheaper models.

Architectural Differences and Model Performance

The results suggest that the differences in architectural designs cause the differences in performance. The performance of DeepLab supports the hypothesis that the multi-scale feature learning from the ASPP is better applicable on burial mound segmentation than the normal bottleneck. This is in line with earlier works (Chen et al, 2016; Chen et al, 2017; Chen et al, 2018). In contrast, the expectation that transformer-based models would outperform Unet was not supported.

Both TransUNet and DenseTrans performed worse than Unet and DeepLab, indicating that transformers are less applicable in this application. This could be due to a number of reasons.

One of the possible explanations is that for segmentation, the more global context does not add to the performance. Burial mounds are small local features, and using only local features might be better fitted than global features through transformers.

Another possibility is that the training dataset was too small, and the transformers were insufficiently trained to start performing. Transformers require significantly more

training data to start performing competitively compared to standard convolutional networks (Dosovitskiy et al., 2020; Kumar, 2025; ZongRen et al., 2023).

A third explanation is that the smaller model implementation hurts performance, and that using the larger proposed models would have increased the performance and stability significantly. Both TransUNet and DenseTrans had transformers removed from the proposed models to accommodate for limited GPU resources.

Computational Costs

The results suggest that transformer-based models are slower at training than convolutional networks. This could be due to transformers containing self-attention mechanisms that require large calculations that take up more time than regular convolutions (Dosovitskiy et al., 2020; Kumar, 2025).

The results further indicate that deeper models, like TransUNets ResNet50 style and DeepLab's Xception-65 style encoder, take up considerably more space than simpler models.

The results also indicate that transformer-based models have higher FLOPs than convolutional networks. This could also be due to the self-attention mechanisms, just like for training time. Training time and FLOPs seem correlated in the results. This could be due to the computations being the main element that takes up time in training the models.

Comparing Performance Metrics

To be able to determine the best model, the importance of the performance metrics should first be determined. IoU and DSC are very similar in the way they both quantify the overlap between predicted segmentation and ground truth segmentation. Their main difference is that IoU punishes false positives and false negatives harsher, improving delineation of burial mounds (Marrakchi et al., 2021).

Similarly, precision was deemed more important than recall, as it prioritizes having correct predictions over not missing any mounds. Burial mound research can be time-consuming and costly (Bureau voor Archeologie, n.d.). A model that produces less wrong predictions saves a lot of resources, and that adds to why DeepLab is most suitable for this application.

Effect of Input Channels

The four compared input channels are DTM, hillshade, slope, and SLRM. From these four, SLRM added the most value, as it is part of all the best combinations, for each model and each metric. SLRM is specifically made to highlight local variations in relief, and burial mounds fit this description. SLRM also is absent in all the worst combinations, adding to the importance of including it as an input layer. Slope is the second-most frequent channel to be featured in the best models, indicating that local gradients are valuable for this application of segmentation. DTM is in a lot of the best combinations, but alone it makes it the worst combination across three of the models, and it is part of the worst combinations for the fourth model as well. Hillshade is featured in a few best combinations and worst combinations. This makes that hillshade would be the first of the four channels to omit, as this channel does not seem to affect the performance much in either direction.

Implications

Out of the four models compared, Unet, TransUNet, DeepLab, and DenseTrans, DeepLab is the most reliable architecture of the four models compared for the application of burial mound segmentation.

For burial mound segmentation, using a Simple Local Relief Model provides the best performance of all four compared input channels. The other channels, Digital Terrain Model, hillshade, and slope, are less important to include but can influence performance. For DeepLab, using only SLRM or together with DTM returns the best performance in terms of intersection over union and precision.

That DeepLab and Unet outperform TransUNet and DenseTrans implies that for this application simpler architectures are competitive with transformer-based models.

Although DeepLab is computationally more expensive than Unet, the added costs are still very small compared to the time and other costs of manually verifying whether segmented burial mounds are actually burial mounds. This justifies using a slightly more expensive model that improves the precision for this application.

Limitations

This research comes with a number of limitations that can influence the findings. One of the limitations is the small training dataset that is used. In total, only 3176 tiles were used for training, validating and testing. A small dataset means less data to optimize the models on, which affects transformer-based models more (Dosovitskiy et al., 2020; Kumar, 2025; ZongRen et al., 2023).

Another limitation is that the models were altered to be able to run on limited graphical processing resources. If feasible, TransUNet and DenseTrans would have had more transformers, and DeepLab would have had a larger encoder, to be closer to the actual proposed models.

Next to that, the models also were all trained using the same hyperparameters. These hyperparameters can be more beneficial for some models than for others, creating a bias for the performance of the four models (Bengio, 2012).

The following limitation is the use of only four architectures. By comparing only four models, these models can seem relatively good or bad at segmenting burial mounds. Adding more models to the comparison could reveal patterns about these models. For example, by comparing with other models, these four could be all clearly outperformed. Then it would not make sense to consider any of these four for this application.

Another limitation is that the input channels may not be optimal for different landscapes. The areas used for training and testing contain a lot of variation of elevation and vegetation, next to the different soil compositions (Kokalj, 2025).

The FLOPs of DenseTrans could not be determined, which makes any inference about transformer-based models requiring more computations less supported, as only one transformer-based model has been measured.

Recommendations

This research has a number of limitations, and therefore also a number of recommendations for future research. First is the use of larger datasets. This might improve the transformer-based models especially, as transformers need a lot of training data to start performing well.

Another recommendation is to investigate newer architectures. There are many developments being made recently and many different architectural approaches are being proposed (Bao et al. 2025; Carion et al., 2025; Kirillov et al., 2023). By comparing these, better suited architectures may be found and deeper insight in what design choices improve performance may be obtained.

Something else that can be researched further is the use of different input channels. There are many ways to process lidar data, and there are many other ways to gather data as well (Conyers, 2012; Conyers, 2023; Verbovšek et al., 2019).

At last, the model transferability can be researched. If the model is in fact well suited to be used for other archaeological features, these can be researched together. It would also be interesting to see if the training data for other features can be combined with the training data for burial mounds to improve performance for burial mound segmentation.

Conclusion

The aim of this study was to compare four deep learning models for burial mound segmentation using lidar data. Next to that, this study also aimed to compare the effect of fifteen combinations of four different input channels on the performance of the models. By doing so, this study contributes to the literature about deep learning models for burial mound segmentation.

Out of the four models, DeepLab had the best performance. The multi-scale feature learning enabled by the ASPP seems very effective for segmenting burial mounds.

The Transformer-based models TransUNet and DenseTrans did not outperform the convolutional models Unet and DeepLab. These findings demonstrate that transformer architectures are not necessarily better for segmenting small archaeological structures.

The SLRM was the most valuable input channel and it consistently improved performance. The comparison of input channels shows that the choice of input channel can be as important as choice of model, as it strongly influences the performance.

Precision and IoU show that DeepLab has the best trade-off between correct and incorrect positive predictions.

DeepLab is computationally more expensive than Unet, but with the added precision and IoU, less falsely segmented areas would need to be manually verified, saving more time and resources.

For detection of new burial mounds using remote sensing, DeepLab should be considered the better choice and transformer-based models should be avoided.

This study was limited by a small dataset to train and test models on, simplified models due to limited GPU resources, the use of identical hyperparameters for all models, and a limited set of architectures and input channels considered.

Future research should evaluate larger datasets, more architectures, and additional input channels. Future research should also investigate how well the models would perform on other archaeological features for transferability.

Overall, this study demonstrates that DeepLab, combined with SLRM as input, is the best suited of all evaluated deep learning models for segmentation of burial mounds using lidar-derived data.

Code availability

All code used to train, test, and evaluate the models used for this thesis can be found on https://github.com/YenotyEtoKruto/KIThesis.

References

Actueel Hoogtebestand Nederland. (2025). *Actueel Hoogtebestand Nederland* (Version 4) [Data set]. https://www.ahn.nl/

Argyrou, A., & Agapiou, A. (2022). A Review of Artificial Intelligence and Remote Sensing for Archaeological Research. *Remote Sensing*, *14*(23), 6000. https://doi.org/10.3390/rs14236000

Bailey, D. (2002). The archaeology of burial mounds. Theory and interpretation. *Pratiques funeraires dans lEurope des XIII–IV s. av. JC*, 23-27.

Bao, M., Lyu, S., Xu, Z., Zhao, Q., Zeng, C., Bai, W., & Cheng, G. (2025, March 25). *ASP-VMUNet: Atrous Shifted Parallel Vision Mamba U-Net for Skin Lesion Segmentation*. arXiv.org. https://arxiv.org/abs/2503.19427

Bengio, Y. (2012, June 24). *Practical recommendations for gradient-based training of deep architectures*. arXiv.org. https://arxiv.org/abs/1206.5533

Bureau voor Archeologie. (n.d.). *Bureau voor Archeologie*. Bureau Voor Archeologie. https://www.bureauvoorarcheologie.nl/

Cambridge University Press and Assessment. (n.d.). Remote sensing. In *Cambridge Dictionary*. Retrieved 05-2026, from https://dictionary.cambridge.org/dictionary/english/remote-sensing

Carion, N., Gustafson, L., Hu, Y., Debnath, S., Hu, R., Suris, D., Ryali, C., Alwala, K. V., Khedr, H., Huang, A., Lei, J., Ma, T., Guo, B., Kalla, A., Marks, M., Greer, J., Wang, M., Sun, P., Rädle, R., . . . Feichtenhofer, C. (2025, November 20). *SAM 3: Segment Anything with Concepts*. arXiv.org. https://arxiv.org/abs/2511.16719

Chen, J., Mei, J., Li, X., Lu, Y., Yu, Q., Wei, Q., Luo, X., Xie, Y., Adeli, E., Wang, Y., Lungren, M. P., Zhang, S., Xing, L., Lu, L., Yuille, A., & Zhou, Y. (2024). TransUNet: Rethinking the U-Net architecture design for medical image segmentation through the lens of transformers. *Medical Image Analysis*, *97*, 103280. https://doi.org/10.1016/j.media.2024.103280

Chen, L., Papandreou, G., Kokkinos, I., Murphy, K., & Yuille, A. L. (2016, June 2). *DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs*. arXiv.org. https://arxiv.org/abs/1606.00915

Chen, L., Papandreou, G., Schroff, F., & Adam, H. (2017, June 17). *Rethinking Atrous convolution for semantic image segmentation*. arXiv.org. https://arxiv.org/abs/1706.05587

Chen, L., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018, February 7). *Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation*. arXiv.org. https://arxiv.org/abs/1802.02611

Chollet, F. (2016). Xception: Deep Learning with Depthwise Separable Convolutions. *arXiv.org*. https://arxiv.org/abs/1610.02357

Collis, R. (1969). Lidar. In *Advances in geophysics* (pp. 113–139). https://doi.org/10.1016/s0065-2687(08)60509-9

Conyers, L. B. (2012). *Interpreting ground-penetrating radar for archaeology*. Left Coast Press.

Conyers, L. B. (2023). *Ground-Penetrating Radar for Archaeology*. Rowman & Littlefield Publishers.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020, October 22). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv.org. https://arxiv.org/abs/2010.11929

Fontijn, D., Bourgeois, Q., Press, S., Louwen, A., & Kok, R. Living near the Dead. The barrow excavations of Rhenen-Elst: two millennia of burial and habitation on the Utrechtse Heuvelrug.

Guyot, A., Lennon, M., Lorho, T., & Hubert-Moy, L. (2021). Combined Detection and Segmentation of Archeological Structures from LiDAR Data Using a Deep Learning Approach. *Journal Of Computer Applications in Archaeology*, *4*(1), 1. https://doi.org/10.5334/jcaa.64

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *2016 IEEE Conference On Computer Vision And Pattern Recognition (CVPR)*, 770-778. https://doi.org/10.1109/cvpr.2016.90

Horn, B. (1981). Hill shading and the reflectance map. *Proceedings of the IEEE*, *69*(1), 14–47. https://doi.org/10.1109/proc.1981.11918

Hupperetz, W., Lammers-Keijsers, Y., Steiner, M., & Vredenburg, J. (2009). Van bodemvondst tot database: handboek voor de amateurarcheoloog. *UvA-DARE (University of Amsterdam)*.

Inomata, T. (2024). Lidar, Space, and Time in Archaeology: Promises and Challenges. *Annual Review Of Anthropology*, *53*(1), 75–92. https://doi.org/10.1146/annurev-anthro-041222-093758

Karpińska, K. (2026). Blood on Ashes: Unburnt Domestic Chicken Remains in Viking Age Cremation Graves. In *Brepols eBooks* (pp. 185–200). https://doi.org/10.1484/m.tane-eb.5.151513

Kato, S., & Hotta, K. (2023). Adaptive t-vMF dice loss: An effective expansion of dice loss for medical image segmentation. *Computers in Biology And Medicine*, *168*, 107695. https://doi.org/10.1016/j.compbiomed.2023.107695

Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W., Dollár, P., & Girshick, R. (2023, April 5). *Segment anything*. arXiv.org. https://arxiv.org/abs/2304.02643

Kochkarev, A., Khvostikov, A., Korshunov, D., Krylov, A., Boguslavskiy, M. (2020). Data balancing method for training segmentation neural networks. *CEUR Workshop Proceedings, 2744*. https://ceur-ws.org/Vol-2744/short19.pdf

Kokalj, Ž. (2025). Standardizing Visualization in Ancient Maya Lidar Research: Techniques, challenges and recommendations. *Archaeological Prospection*, *32*(4), 967–988. https://doi.org/10.1002/arp.70002

Kumar, S. (2025). Advancements in medical image segmentation: A review of transformer models. *Computers & Electrical Engineering*, *123*, 110099. https://doi.org/10.1016/j.compeleceng.2025.110099

Maghsoud, E. M., Koochakzaei, A., & Naseri, R. (2026). Stable isotope analysis reveals Bronze Age diets at Deh Dumen Iran. *Scientific Reports*. https://doi.org/10.1038/s41598-026-49529-5

Mäkinen, T., Maijanen, H., & Seitsonen, O. (2026). Using a pet cemetery as a control study to optimise clandestine burial search in the Nordic region with insights into seasonal variation in vegetation, topography and temperature. *Forensic Science International*, *381*, 112827. https://doi.org/10.1016/j.forsciint.2026.112827

Marrakchi, Y., Makansi, O., & Brox, T. (2021). Fighting Class Imbalance with Contrastive Learning. In *Lecture notes in computer science* (pp. 466–476). https://doi.org/10.1007/978-3-030-87199-4_44

Miller, D. S., & Rivera, J. D. (2006). Hallowed Ground, Place, and Culture. *Space And Culture*, *9*(4), 334–350. https://doi.org/10.1177/1206331206292450

Oltean, I. A. (2013). Burial mounds and settlement patterns: a quantitative approach to their identification from the air and interpretation. *Antiquity*, *87*(335), 202-219.

Ramyachitra, D., & Manikandan, P. (2014). IMBALANCED DATASET CLASSIFICATION AND SOLUTIONS: A REVIEW. In Bharathiar University, *International Journal Of*

Computing And Business Research (IJCBR) (Vol. 5, Nummer 4). https://researchmanuscripts.com/July2014/2.pdf

Rijksdienst voor het Cultureel Erfgoed. (2018-present). *Grafheuvels* [Map]. Retrieved 05-2026, from https://nationaalgeoregister.nl/geonetwork/srv/dut/catalog.search#/metadata/6eafac71-b2c4-42a6-be65-3f97510081a7

Ronneberger, O. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *arXiv (Cornell University)*, 234–241. https://doi.org/10.48550/arxiv.1505.04597

Taciuc, I., Dumitru, M., Marinescu, A., Serboiu, C., Musat, G., Gherghe, M., Costache, A., & Vrinceanu, D. (2025). Enhancing Malignant Lymph Node Detection in Ultrasound Imaging: A Comparison Between the Artificial Intelligence Accuracy, Dice Similarity Coefficient and Intersection over Union. *Journal of Mind and Medical Sciences*, 12(1), 29. https://doi.org/10.3390/jmms12010029

Taghanaki, S. A., Zheng, Y., Zhou, S. K., Georgescu, B., Sharma, P., Xu, D., Comaniciu, D., & Hamarneh, G. (2019). Combo loss: Handling input and output imbalance in multi-organ segmentation. *Computerized Medical Imaging And Graphics*, *75*, 24–33. https://doi.org/10.1016/j.compmedimag.2019.04.005

Verbovšek, T., Popit, T., & Kokalj, Ž. (2019). VAT Method for Visualization of Mass Movement Features: An alternative to hillshaded DEM. *Remote Sensing*, *11*(24), 2946. https://doi.org/10.3390/rs11242946

Victor, N., Maddikunta, P. K. R., Mary, D. R. K., Murugan, R., Chengoden, R., Gadekallu, T. R., Rakesh, N., Zhu, Y., & Paek, J. (2024). Remote Sensing for Agriculture in the Era of Industry 5.0—A Survey. *IEEE Journal Of Selected Topics in Applied Earth*

Observations And Remote Sensing, *17*, 5920–5945. https://doi.org/10.1109/jstars.2024.3370508

Vinci, G., Vanzani, F., Fontana, A., & Campana, S. (2024). LiDAR Applications in Archaeology: A Systematic review. *Archaeological Prospection*, *32*(1), 81–101. https://doi.org/10.1002/arp.1931

Wicke, C. R. (1965). Pyramids and Temple Mounds: Mesoamerican Ceremonial Architecture in Eastern North America. *American Antiquity*, *30*(4), 409–420. https://doi.org/10.2307/277940

Wu, M., Wei, G., Wu, X., & Xia, F. (2026). Centralized control and local response: archaeometallurgical evidence from the Nanping cemetery in Changde, Hunan during the Western Han Dynasty. *The European Physical Journal Plus*, *141*(3). https://doi.org/10.1140/epjp/s13360-026-07353-w

Yang, H. (2025). Segmenting ancient cemeteries under forests using synthesized LiDAR-derived data and deep convolutional neural network. *Npj Heritage Science*, *13*(1). https://doi.org/10.1038/s40494-025-01798-5

Zakšek, K., Oštir, K., & Kokalj, Ž. (2011). Sky-View Factor as a relief visualization technique. *Remote Sensing*, *3*(2), 398–415. https://doi.org/10.3390/rs3020398

ZongRen, L., Silamu, W., Yuzhen, W., & Zhe, W. (2023). DenseTrans: Multimodal brain tumor segmentation using SWIN Transformer. *IEEE Access*, *11*, 42895–42908. https://doi.org/10.1109/access.2023.3272055

Appendix A

Performance Results per Model

Table A1

Performance Results for Unet

Input Channels	IoU	DSC	Precision	Recall
DTM	0.327	0.493	0.516	0.471
Hillshade	0.471	0.640	0.746	0.561
Slope	0.512	0.677	0.738	0.626
SLRM	0.576	0.731	0.789	0.681
DTM + Hillshade	0.482	0.650	0.769	0.563
DTM + Slope	0.517	0.681	0.790	0.599
DTM + SLRM	0.560	0.718	0.772	0.671
Hillshade + Slope	0.536	0.698	0.781	0.630
Hillshade + SLRM	0.542	0.703	0.736	0.673
Slope + SLRM	0.573	0.729	0.784	0.681
DTM + Hillshade + Slope	0.536	0.698	0.765	0.641
DTM + Hillshade + SLRM	0.537	0.699	0.742	0.660
DTM + Slope + SLRM	0.577	0.732	0.816	0.663
Hillshade + Slope + SLRM	0.561	0.719	0.790	0.660
DTM + Hillshade + Slope + SLRM	0.565	0.722	0.775	0.676

Table A2

Performance Results for TransUNet

Input Channels	IoU	DSC	Precision	Recall
DTM	0.049	0.093	0.052	0.468
Hillshade	0.430	0.601	0.701	0.526
Slope	0.497	0.664	0.736	0.604
SLRM	0.520	0.684	0.658	0.712
DTM + Hillshade	0.460	0.631	0.745	0.547
DTM + Slope	0.414	0.586	0.712	0.498
DTM + SLRM	0.494	0.662	0.675	0.649
Hillshade + Slope	0.262	0.415	0.334	0.549
Hillshade + SLRM	0.506	0.672	0.781	0.590
Slope + SLRM	0.591	0.743	0.716	0.772
DTM + Hillshade + Slope	0.348	0.517	0.471	0.572
DTM + Hillshade + SLRM	0.528	0.691	0.711	0.672
DTM + Slope + SLRM	0.504	0.670	0.626	0.720
Hillshade + Slope + SLRM	0.522	0.686	0.699	0.674
DTM + Hillshade + Slope + SLRM	0.515	0.680	0.711	0.652

Table A3

Performance Results for DeepLab

Input Channels	IoU	DSC	Precision	Recall
DTM	0.425	0.597	0.729	0.505
Hillshade	0.472	0.641	0.803	0.534
Slope	0.530	0.693	0.823	0.598
SLRM	0.556	0.714	0.867	0.607
DTM + Hillshade	0.467	0.637	0.785	0.535
DTM + Slope	0.475	0.644	0.852	0.517
DTM + SLRM	0.571	0.727	0.815	0.656
Hillshade + Slope	0.520	0.684	0.797	0.599
Hillshade + SLRM	0.567	0.724	0.796	0.664
Slope + SLRM	0.543	0.703	0.791	0.663
DTM + Hillshade + Slope	0.530	0.693	0.837	0.591
DTM + Hillshade + SLRM	0.564	0.722	0.822	0.643
DTM + Slope + SLRM	0.557	0.716	0.800	0.647
Hillshade + Slope + SLRM	0.569	0.725	0.826	0.646
DTM + Hillshade + Slope + SLRM	0.566	0.723	0.823	0.644

Table A4

Performance Results of DenseTrans

Input Channels	IoU	DSC	Precision	Recall
DTM	0.492	0.659	0.714	0.613
Hillshade	0.450	0.620	0.660	0.585
Slope	0.474	0.644	0.671	0.618
SLRM	0.523	0.687	0.735	0.645
DTM + Hillshade	0.419	0.590	0.621	0.563
DTM + Slope	0.486	0.654	0.746	0.583
DTM + SLRM	0.499	0.666	0.716	0.622
Hillshade + Slope	0.470	0.640	0.703	0.587
Hillshade + SLRM	0.500	0.667	0.673	0.661
Slope + SLRM	0.530	0.693	0.699	0.686
DTM + Hillshade + Slope	0.450	0.620	0.719	0.545
DTM + Hillshade + SLRM	0.497	0.664	0.670	0.658
DTM + Slope + SLRM	0.525	0.688	0.765	0.625
Hillshade + Slope + SLRM	0.519	0.683	0.687	0.679
DTM + Hillshade + Slope + SLRM	0.527	0.691	0.699	0.682

Appendix B

Computational Costs per Model

Table B1

Computational Costs for Unet

Number of Input Channels	Training time (s)	Model Size (MB)	Number of Parameters	GFLOPs
1	1024.14 ± 1.21	22.416	1940817	6.04
2	1033.56 ± 3.22	22.418	1940961	6.06
3	1046.43 ± 2.83	22.420	1941105	6.08
4	1083.42	22.422	1941249	6.10

Table B2

Computational Costs for TransUNet

Number of Input Channels	Training time (s)	Model Size (MB)	Number of Parameters	GFLOPs
1	3007.14 ± 2.16	190.24	16543681	20.41
2	3015.58 ± 4.87	190.28	16546817	20.51
3	3024.40 ± 5.70	190.31	16549953	20.61
4	3075.08	190.34	16553089	20.72

Table B3

Computational Costs for DeepLab

Number of Input Channels	Training time (s)	Model Size (MB)	Number of Parameters	GFLOPs
1	2106.20 ± 14.26	285.82	24933081	14.89
2	2108.04 ± 10.09	285.82	24933369	14.89
3	2100.90 ± 7.27	285.82	24933657	14.90
4	2146.86	285.83	24933945	14.91

Table B4

Computational Costs for DenseTrans

Number of Input Channels	Training time (s)	Model Size (MB)	Number of Parameters	GFLOPs
1	4148.59 ± 8.03	17.813	1514123	—
2	4157.44 ± 6.78	17.813	1514149	—
3	4188.38 ± 3.07	17.814	1514175	—
4	4200.45	17.813	1514201	—

Note. — = Due to the implementation of the Swin transformers in DenseTrans, standard functions did not work on this model to decide FLOPs.

Appendix C

AI Disclosure Statement

I, Friso Harff (7526946), hereby confirm that the present work is my original work. I declare I have used AI tools only as allowed for this work. I acknowledge my responsibility as a student/learner to thoroughly verify all outputs and content produced by AI tools and accept full accountability for their accuracy and validity.

Signature of the student:



Date: June 26, 2026